Contents

Preparing the Project

Step 1: Scaffold the Project

```
npm create vite@latest react-props-lesson -- --template react-ts
cd react-props-lesson
npm install
```

Step 2: Set Up the File Structure

src/
- components/
  - UserCard.tsx
  - StatusBadge.tsx
  - ActionButton.tsx
  - TaskList.tsx
  - PageLayout.tsx
- App.tsx
- main.tsx

More on Using Props

1. The User Profile Card (Basic Data Props)

A reusable card to display user information on a dashboard or directory.

src/components/UserCard.tsx

```
import React from 'react';

// 1. Define the shape of the props
interface UserCardProps {
  name: string;
  role: string;
}

// 2. Destructure the props in the component signature
const UserCard = ({ name, role }: UserCardProps) => {
  return (
    <div style={{ border: '1px solid #ccc', padding: '10px', borderRadius: '5px' }}>
      <h3>{name}</h3>
      <p style={{ color: 'gray' }}>{role}</p>
    </div>
  );
};

export default function App() {
  return (
    <div>
      <UserCard name="Alice Johnson" role="Software Engineer" />
      <UserCard name="Marcus Chen" role="Product Manager" />
    </div>
```

```
  );
}
```

2. The Status Badge (Union Types for Strictness)
A small colored badge used on an e-commerce site to show order status.

src/components/StatusBadge.tsx

```
import React from 'react';

// 1. Restrict the status to only three specific strings
interface BadgeProps {
  status: 'pending' | 'shipped' | 'delivered';
}

const StatusBadge = ({ status }: BadgeProps) => {
  // 2. Use the prop to drive dynamic styles or classes
  const getBackgroundColor = () => {
    if (status === 'pending') return 'orange';
    if (status === 'shipped') return 'blue';
    return 'green'; // delivered
  };

  return (
    <span style={{ backgroundColor: getBackgroundColor(), color: 'white', padding: '4px 8px' }}>
      {status.toUpperCase()}
    </span>
  );
};

export default function App() {
  return (
    <div>
      <StatusBadge status="pending" />
      {/* <StatusBadge status="lost" /> <-- TypeScript will throw a helpful error here! */}
    </div>
  );
}
```

3. The Reusable Action Button (Functions and Optional Props)
How to pass behavior (functions) down to child components, alongside introducing **optional props** with the ? syntax.

A standardized button used across an application that needs to trigger different actions and can occasionally be disabled.

src/components/ActionButton.tsx

```
import React from 'react';

// 1. Define a function prop and an optional boolean
interface ActionButtonProps {
  label: string;
  onClick: () => void;
  isDisabled?: boolean; // The '?' makes this optional
}

// 2. Provide a default value (false) for the optional prop
const ActionButton = ({ label, onClick, isDisabled = false }: ActionButtonProps) => {
  return (
    <button onClick={onClick} disabled={isDisabled}>
      {label}
    </button>
  );
};

export default function App() {
  const handleSave = () => alert("Saved successfully!");

  return (
    <div>
      <ActionButton label="Save Changes" onClick={handleSave} />
      <ActionButton label="Delete Account" onClick={() => alert("Are you sure?")} isDisabled={true} />
    </div>
  );
}
```

Creating a Page Layout for the Prop Examples

src/components/PageLayout.tsx

```
import React from 'react';

interface PageLayoutProps {
  title: string;
  children: React.ReactNode;
}

export default function PageLayout({ title, children }: PageLayoutProps) {
  return (
    <div style={{ backgroundColor: '#f9f9f9', padding: '20px', borderRadius: '8px' }}>
      <h2>{title}</h2>
      <hr style={{ borderColor: '#ddd' }} />
      <div style={{ marginTop: '20px' }}>
        {children}
      </div>
```

```
  </div>
 );
}
```

src/App.tsx

```
import { useState } from 'react';
import UserCard from './components/UserCard';
import StatusBadge from './components/StatusBadge';
import ActionButton from './components/ActionButton';
import PageLayout from './components/PageLayout';

// Define a type for our tabs to prevent typos
type TabName = 'Profile' | 'E-commerce' | 'Settings';

export default function App() {
 // State to track which lesson we are viewing
 const [activeTab, setActiveTab] = useState<TabName>('Profile');

 return (
  <div style={{ padding: '40px', fontFamily: 'sans-serif', maxWidth: '800px', margin: '0 auto' }}>
   <h1>React Props Playground</h1>

   {/* Navigation Menu */}
   <div style={{ display: 'flex', gap: '10px', marginBottom: '20px' }}>
    {(['Profile', 'E-commerce', 'Settings'] as TabName[]).map((tab) => (
     <button
      key={tab}
      onClick={() => setActiveTab(tab)}
      style={{ fontWeight: activeTab === tab ? 'bold' : 'normal', padding: '8px 16px' }}
     >
      {tab} Example
     </button>
    ))}
   </div>

   {/* Conditionally Render the Layouts based on the active tab */}
   {activeTab === 'Profile' && (
    <PageLayout title="User Directory">
     <div style={{ display: 'flex', gap: '15px' }}>
      <UserCard name="Alice Johnson" role="Software Engineer" />
      <UserCard name="Marcus Chen" role="Product Manager" />
     </div>
    </PageLayout>
   )}

   {activeTab === 'E-commerce' && (
    <PageLayout title="Recent Orders">
     <div style={{ display: 'flex', flexDirection: 'column', gap: '15px' }}>
      <p>Order #101: <StatusBadge status="delivered" /></p>
```

```
        <p>Order #102: <StatusBadge status="shipped" /></p>
        <p>Order #103: <StatusBadge status="pending" /></p>
      </div>
    </PageLayout>
   )}

   {activeTab === 'Settings' && (
    <PageLayout title="Account Actions">
      <div style={{ display: 'flex', gap: '15px' }}>
       <ActionButton label="Save Changes" onClick={() => alert('Saved!')} />
       <ActionButton label="Delete Account" onClick={() => alert('Action blocked.')} isDisabled={true}
/>
      </div>
    </PageLayout>
   )}
  </div>
 );
}
```

State Management using Redux (Redux Toolkit)

The Installation

```
npm install @reduxjs/toolkit react-redux
```

Create the Slices (The State)

A "slice" contains the initial state, the reducers (how the state changes), and the actions, all in one file. We will create two slices for our two real-world examples: Authentication and a Shopping Cart.

1. **The Auth Slice (src/features/authSlice.ts)**
   Manages global login state to avoid prop-drilling.

```
import { createSlice, PayloadAction } from '@reduxjs/toolkit';

interface AuthState {
 isAuthenticated: boolean;
 username: string | null;
}

const initialState: AuthState = {
 isAuthenticated: false,
 username: null,
};

const authSlice = createSlice({
 name: 'auth',
 initialState,
```

```
  reducers: {
   login: (state, action: PayloadAction<string>) => {
    state.isAuthenticated = true;
    state.username = action.payload;
   },
   logout: (state) => {
    state.isAuthenticated = false;
    state.username = null;
   },
  },
});

export const { login, logout } = authSlice.actions;
export default authSlice.reducer;
```

2. **The Cart Slice (src/features/cartSlice.ts)**
   Manages an array of items. Notice how RTK allows us to use .push() without manually spreading arrays.

```
import { createSlice, PayloadAction } from '@reduxjs/toolkit';

interface CartItem {
  id: string;
  name: string;
}

const cartSlice = createSlice({
  name: 'cart',
  initialState: [] as CartItem[],
  reducers: {
   addToCart: (state, action: PayloadAction<CartItem>) => {
    state.push(action.payload);
   },
   removeFromCart: (state, action: PayloadAction<string>) => {
    return state.filter(item => item.id !== action.payload);
   }
  }
});

export const { addToCart, removeFromCart } = cartSlice.actions;
export default cartSlice.reducer;
```

Configure the Store (The Brain)

Combine those slices into a single global store. This is also where we define standard TypeScript types so our app knows exactly what data exists.

Create **src/store.ts**:

```
import { configureStore } from '@reduxjs/toolkit';
import authReducer from './features/authSlice';
import cartReducer from './features/cartSlice';

export const store = configureStore({
  reducer: {
    auth: authReducer,
    cart: cartReducer,
  },
});

// Export standard types for TypeScript to use throughout the app
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

Step 4: Provide the Store (The Wiring)
For the React app to actually see the Redux store, you must wrap your entire application in a Redux <Provider>.

Update your **src/main.tsx** (or index.tsx depending on your setup):

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import { Provider } from 'react-redux';
import { store } from './store';

ReactDOM.createRoot(document.getElementById('root') as HTMLElement).render(
  <React.StrictMode>
    {/* Wrap the App component with the Redux Provider */}
    <Provider store={store}>
      <App />
    </Provider>
  </React.StrictMode>
);
```

Step 5: Consume the State in Components
Now that the app is wired up, we use useSelector to read data, and useDispatch to trigger actions.

Example 1 Component: The Navbar (src/components/Navbar.tsx)

```
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { RootState } from '../store';
import { login, logout } from '../features/authSlice';
```

```
export default function Navbar() {
 const dispatch = useDispatch();
 // Read from the global store using our RootState type
 const { isAuthenticated, username } = useSelector((state: RootState) => state.auth);

 return (
  <nav style={{ padding: '15px', background: '#333', color: 'white' }}>
   {isAuthenticated ? (
    <div>
     <span>Welcome, {username}! </span>
     <button onClick={() => dispatch(logout())}>Log Out</button>
    </div>
   ) : (
    <button onClick={() => dispatch(login('JaneDoe'))}>Log In</button>
   )}
  </nav>
 );
}
```

Example 2 Component: The Product Page (src/components/ProductPage.tsx)

```
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { RootState } from '../store';
import { addToCart, removeFromCart } from '../features/cartSlice';

export default function ProductPage() {
 const dispatch = useDispatch();
 const cartItems = useSelector((state: RootState) => state.cart);

 const handleBuy = () => {
  // Generate a quick random ID for the example
  const newItem = { id: Date.now().toString(), name: 'Mechanical Keyboard' };
  dispatch(addToCart(newItem));
 };

 return (
  <div style={{ padding: '20px' }}>
   <h2>Store</h2>
   <button onClick={handleBuy}>Add Keyboard to Cart</button>

   <div style={{ marginTop: '20px', borderTop: '1px solid #ccc' }}>
    <h3>Cart ({cartItems.length} items)</h3>
    <ul>
     {cartItems.map((item) => (
      <li key={item.id}>
       {item.name}
       <button onClick={() => dispatch(removeFromCart(item.id))} style={{ marginLeft: '10px' }}>
        Remove
```

```
          </button>
        </li>
      ))}
    </ul>
   </div>
  </div>
 );
}
```

React Hooks and Custom Hooks

Essential React Hooks

1. useEffect: Fetching Data on Mount
This component fetches user data from a public API the moment it renders on the screen.

src/components/UserProfile.tsx

```
import { useState, useEffect } from 'react';

interface User {
 name: string;
 email: string;
}

export default function UserProfile() {
 const [user, setUser] = useState<User | null>(null);

 // The empty array [] ensures this runs exactly once when the component mounts
 useEffect(() => {
  fetch('https://jsonplaceholder.typicode.com/users/1')
   .then((res) => res.json())
   .then((data) => setUser(data));
 }, []);

 if (!user) return <p>Loading profile...</p>;

 return (
  <div style={{ padding: '15px', border: '1px solid #ccc', borderRadius: '5px' }}>
   <h3>{user.name}</h3>
   <p>{user.email}</p>
  </div>
 );
}
```

2. useRef: The Auto-Focusing Search Bar
This component uses a reference to directly interact with a DOM element (the input field) without triggering a re-render.

src/components/SearchBar.tsx

```
import { useRef } from 'react';

export default function SearchBar() {
 // Create a reference attached to an HTML Input
 const inputRef = useRef<HTMLInputElement>(null);

 const handleFocus = () => {
  // Directly trigger the browser's native focus method
  if (inputRef.current) {
   inputRef.current.focus();
  }
 };

 return (
  <div style={{ padding: '15px', border: '1px solid #ccc', borderRadius: '5px' }}>
   <input
    ref={inputRef}
    type="text"
    placeholder="Search..."
    style={{ padding: '8px', marginRight: '10px' }}
   />
   <button onClick={handleFocus}>Focus Input</button>
  </div>
 );
}
```

Custom Hooks

1. useToggle: The Boilerplate Killer
A reusable hook to handle true/false states (like opening/closing menus).

src/hooks/useToggle.ts

```
import { useState } from 'react';

export function useToggle(initialValue: boolean = false) {
 const [value, setValue] = useState(initialValue);
 const toggle = () => setValue((prev) => !prev);

 return [value, toggle] as const;
}
```

src/components/DropdownMenu.tsx

```
import { useToggle } from '../hooks/useToggle';

export default function DropdownMenu() {
 // Using our custom hook cleans up the component logic instantly
 const [isOpen, toggleOpen] = useToggle(false);

 return (
  <div style={{ padding: '15px', border: '1px solid #ccc', borderRadius: '5px' }}>
   <button onClick={toggleOpen}>
    {isOpen ? 'Close Menu' : 'Open Menu'}
   </button>

   {isOpen && (
    <ul style={{ marginTop: '10px', background: '#eee', padding: '10px' }}>
     <li>Profile</li>
     <li>Settings</li>
    </ul>
   )}
  </div>
 );
}
```

2. useLocalStorage: The State Saver

This hook syncs standard React state with the browser's local storage so data persists across page refreshes.

src/hooks/useLocalStorage.ts

```
import { useState } from 'react';

export function useLocalStorage<T>(key: string, initialValue: T) {
 const [storedValue, setStoredValue] = useState<T>(() => {
  const item = window.localStorage.getItem(key);
  return item ? JSON.parse(item) : initialValue;
 });

 const setValue = (value: T) => {
  setStoredValue(value);
  window.localStorage.setItem(key, JSON.stringify(value));
 };

 return [storedValue, setValue] as const;
}
```

src/components/ThemeSwitcher.tsx

```
import { useLocalStorage } from '../hooks/useLocalStorage';

export default function ThemeSwitcher() {
  // State is now automatically saved to local storage under the key 'theme'
  const [theme, setTheme] = useLocalStorage<'light' | 'dark'>('theme', 'light');

  const toggleTheme = () => setTheme(theme === 'light' ? 'dark' : 'light');

  return (
    <div style={{
      padding: '15px',
      borderRadius: '5px',
      background: theme === 'light' ? '#fff' : '#333',
      color: theme === 'light' ? '#000' : '#fff',
      border: '1px solid #ccc'
    }}>
      <h3>Current Theme: {theme}</h3>
      <button onClick={toggleTheme}>Switch Theme</button>
    </div>
  );
}
```

CSS Modules, Tailwind CSS, and a Component Library (using Material UI)

CSS Modules

CSS Modules automatically generate unique class names (like _button_1x89f), ensuring your styles never accidentally leak into other components. Vite supports CSS Modules out of the box. Just name your file .module.css.

1. The Alert Banner (Dynamic Styling)
This demonstrates how to pass a prop to dynamically switch CSS classes for different visual states (success vs. error).

src/components/Banner.module.css

```
.banner {
  padding: 15px;
  border-radius: 6px;
  font-weight: bold;
  text-align: center;
  margin-bottom: 10px;
}

.success {
  background-color: #d1fae5;
```

```
  color: #065f46;
  border: 1px solid #34d399;
}

.error {
  background-color: #fee2e2;
  color: #991b1b;
  border: 1px solid #f87171;
}
```

src/components/Banner.tsx

```
import styles from './Banner.module.css';

interface BannerProps {
  message: string;
  type: 'success' | 'error';
}

export default function Banner({ message, type }: BannerProps) {
  // We use bracket notation to dynamically select the class from the module
  return (
    <div className={` ${styles.banner} ${styles[type]}`}>
      {message}
    </div>
  );
}
```

2. The Avatar Card (Scoped Layout)

This shows a practical UI component where CSS Modules keep the internal layout styles completely isolated from the rest of the app.

src/components/AvatarCard.module.css

```
.card {
  display: flex;
  align-items: center;
  gap: 15px;
  padding: 15px;
  background: #f9fafb;
  border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
  max-width: 300px;
}

.avatar {
  width: 50px;
  height: 50px;
  border-radius: 50%;
  background-color: #3b82f6;
```

```
  color: white;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.2rem;
  font-weight: bold;
}

.info h4 { margin: 0 0 4px 0; }
.info p { margin: 0; color: #6b7280; font-size: 0.9rem; }
```

src/components/AvatarCard.tsx

```
import styles from './AvatarCard.module.css';

interface AvatarCardProps {
  name: string;
  role: string;
}

export default function AvatarCard({ name, role }: AvatarCardProps) {
  // Get the first initial for the avatar circle
  const initial = name.charAt(0).toUpperCase();

  return (
    <div className={styles.card}>
      <div className={styles.avatar}>{initial}</div>
      <div className={styles.info}>
        <h4>{name}</h4>
        <p>{role}</p>
      </div>
    </div>
  );
}
```

Tailwind CSS

Utility-first CSS. You build designs directly in your TSX using predefined class names without ever writing custom CSS files.

Install TailwindCSS

```
npm install -D tailwindcss @tailwindcss/vite
```

Update vite.config.ts

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'
```

```
export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
  ],
})
```

Create/Update file *tailwind.config.js* (at the root of your project)

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{ts,tsx,js,jsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

Update existing file *src/index.css* (Clear the file completely and add these 3 lines):

```
Import “tailwindcss”;
```

Check Tailwind CSS Classes here:
https://flowbite.com/tools/tailwind-cheat-sheet/
https://tailwind.build/classes

1. The Responsive Navigation Bar
This demonstrates Tailwind's greatest strength: making things responsive with prefixes like md: (medium screens and up) directly in the markup.

src/components/NavBar.tsx

```
export default function NavBar() {
  return (
    <nav className="bg-gray-800 text-white p-4 flex flex-col md:flex-row justify-between items-center rounded-lg mb-6">
      <div className="text-xl font-bold mb-4 md:mb-0">
        MyApp
      </div>
      <ul className="flex space-x-6">
        <li><a href="#" className="hover:text-blue-400 transition-colors">Home</a></li>
        <li><a href="#" className="hover:text-blue-400 transition-colors">Features</a></li>
        <li><a href="#" className="hover:text-blue-400 transition-colors">Pricing</a></li>
      </ul>
    </nav>
  );
}
```

2. The Interactive Pricing Card
This shows how easily you can add hover states (hover:), shadows, and gradients without touching a CSS file.

src/components/PricingCard.tsx

```
export default function PricingCard() {
 return (
  <div className="max-w-sm p-6 bg-white border border-gray-200 rounded-xl shadow-md
hover:shadow-xl transition-shadow duration-300">
    <h5 className="mb-2 text-2xl font-bold text-gray-900">Pro Plan</h5>
    <p className="mb-4 text-gray-500">Perfect for growing teams and businesses.</p>
    <div className="text-4xl font-extrabold mb-6">$29<span className="text-lg text-gray-500 font-
normal">/mo</span></div>

    <button className="w-full bg-blue-600 hover:bg-blue-700 text-white font-semibold py-2 px-4
rounded-lg transition-colors">
     Upgrade to Pro
    </button>
  </div>
 );
}
```

Component Libraries (Material UI / MUI)

Pre-styled, fully functional, and accessible React components created by Google. You use these to avoid reinventing the wheel for complex UI pieces.

```
npm install @mui/material @emotion/react @emotion/styled @mui/icons-material
```

1. The Data Table (Pagination & Sorting Built-in)
Building a good table from scratch takes hours. MUI gives you a production-ready one in seconds.

src/components/UserTable.tsx

```
import { Table, TableBody, TableCell, TableContainer, TableHead, TableRow, Paper } from
'@mui/material';

const rows = [
 { id: 1, name: 'Alice Johnson', status: 'Active' },
 { id: 2, name: 'Marcus Chen', status: 'Offline' },
 { id: 3, name: 'Sarah Smith', status: 'Active' },
];

export default function UserTable() {
 return (
  <TableContainer component={Paper} elevation={3}>
   <Table aria-label="simple table">
    <TableHead style={{ backgroundColor: '#f5f5f5' }}>
     <TableRow>
      <TableCell><strong>ID</strong></TableCell>
```

```
        <TableCell><strong>Name</strong></TableCell>
        <TableCell align="right"><strong>Status</strong></TableCell>
       </TableRow>
      </TableHead>
      <TableBody>
       {rows.map((row) => (
        <TableRow key={row.id} hover>
         <TableCell>{row.id}</TableCell>
         <TableCell>{row.name}</TableCell>
         <TableCell align="right">{row.status}</TableCell>
        </TableRow>
       ))}
      </TableBody>
     </Table>
   </TableContainer>
  );
}
```

2. The Confirmation Dialog (Modal)
Managing the backdrop, centering, and accessibility of a modal overlay is notoriously difficult. MUI's Dialog handles it perfectly via simple boolean props.

src/components/ConfirmDialog.tsx

```
import { useState } from 'react';
import { Button, Dialog, DialogActions, DialogContent, DialogContentText, DialogTitle } from
'@mui/material';
import DeleteIcon from '@mui/icons-material/Delete';

export default function ConfirmDialog() {
 const [open, setOpen] = useState(false);

 const handleOpen = () => setOpen(true);
 const handleClose = () => setOpen(false);
 const handleConfirm = () => {
  alert("Item Deleted!");
  setOpen(false);
 };

 return (
  <div>
   <Button variant="contained" color="error" startIcon={<DeleteIcon />} onClick={handleOpen}>
    Delete Data
   </Button>

   <Dialog open={open} onClose={handleClose}>
    <DialogTitle>Are you absolutely sure?</DialogTitle>
    <DialogContent>
     <DialogContentText>
      This action cannot be undone. This will permanently delete your data from our servers.
```

```
      </DialogContentText>
    </DialogContent>
    <DialogActions>
      <Button onClick={handleClose} color="primary">Cancel</Button>
      <Button onClick={handleConfirm} color="error" variant="contained">
        Confirm Delete
      </Button>
    </DialogActions>
   </Dialog>
  </div>
 );
}
```